Enunciado (v2): Sistema LogiTech, modele você mesmo

Neste trabalho, o enunciado descreve o problema e as regras de negócio, mas não diz quais classes, interfaces ou padrões usar: definir isso, e justificar, é parte da sua tarefa.

1. Contexto e objetivos

A LogiTech é uma operadora logística fictícia que transporta encomendas por três modais: rodoviário, aéreo e aquaviário. Você vai construir o núcleo de simulação que calcula **custo, tempo de entrega e emissão de CO_2** e mantém os interessados informados sobre o andamento das entregas.

O trabalho avalia quatro competências:

- **análise do problema**: identificar conceitos, responsabilidades e relações a partir de uma descrição em linguagem natural;
- **projeto orientado a objetos**: decidir quais classes e interfaces existem, quem faz o quê e por quê;
- **estratégia de resolução**: escolher e justificar mecanismos de projeto (incluindo padrões GoF, se fizerem sentido) para lidar com variação e evolução;
- **uso crítico de IA**: instruir, revisar e corrigir ferramentas como Claude Code ou Antigravity.

Gerar código com IA é permitido e esperado. Entregar código ou decisões de projeto que você não entende ou não validou não é.

Cronograma de entregas

Etapa	Data	O que entregar
Etapa 1 (seção 8)	**27/11**	Somente o Documento de Projeto, sem código. Ele é usado pelo professor para analisar a estratégia de resolução adotada.
Etapa 2 (seção 9)	**04/12** (dia da apresentação)	Implementação mostrada no dia: código, testes, Documento de Projeto revisado e relatório de uso de IA, seguidos do stress test e da defesa oral (seção 10).

2. O problema

A LogiTech recebe encomendas com peso, distância, origem e destino. Para cada uma, o sistema precisa:

1. escolher o **modal** (rodoviário, aéreo ou aquaviário), cada um com custos, velocidade e emissões próprios;
2. calcular **frete, tempo estimado e CO_2** segundo um **perfil de precificação** (econômico, rápido ou ecológico) que o operador pode trocar a qualquer momento, inclusive com o sistema em execução;
3. acompanhar o **status e a posição** de cada transporte ao longo da entrega;

4. **avisar automaticamente** a central de controle, a auditoria ambiental e o cliente quando o status ou a posição de um transporte mudar, conforme a seção 4.3;
5. processar encomendas em **lote**, lidas da entrada padrão (uma encomenda por linha), rejeitando as inválidas sem interromper o processamento;
6. emitir um **relatório consolidado** (custo total, tempo médio, CO_2 total).

Vocabulário usado neste enunciado (ele fixa os termos, não as classes):

Termo	Significado
Modal	Rodoviário, aéreo ou aquaviário (seção 3.1)
Encomenda	Os dados do pedido: ID, peso, distância, origem, destino e perfil
Transporte	A entrega de uma encomenda em andamento, com status e posição
Perfil	Econômico, rápido ou ecológico (seção 3.3)
Interessado	Quem recebe os avisos: central de controle, auditoria ambiental, cliente e os que surgirem no futuro

3. Regras de negócio

Os valores abaixo são fictícios, mas mantêm proporções realistas entre os modais.

3.1 Parâmetros por modal

Parâmetro	Rodoviário	Aéreo	Aquaviário
Custo por km (R$/km)	1,50	4,00	0,60
Custo por kg (R$/kg)	0,40	2,50	0,15
Velocidade média (km/h)	60	800	30
Tempo fixo de manuseio (h)	2	6	24
Fator de emissão (kg CO_2 por tonelada·km)	0,062	0,602	0,016
Capacidade máxima por encomenda (kg)	20.000	5.000	100.000

3.2 Valores base

Os valores base dependem apenas do modal e da encomenda.

```
freteBase = distanciaKm × custoPorKm + pesoKg × custoPorKg
```

Se `pesoKg > 1000`, o frete base recebe uma sobretaxa de carga pesada: `freteBase = freteBase × 1,05`.

```
tempoBaseH = distanciaKm / velocidadeMedia + tempoManuseio
```

```
co2BaseKg = (pesoKg / 1000) × distanciaKm × fatorEmissao
```

3.3 Perfis de precificação

Cada perfil define como o frete, o tempo e o CO_2 finais são obtidos a partir dos valores base. Os três perfis abaixo fazem isso aplicando multiplicadores. O perfil não depende do modal: as mesmas regras valem para os três.

Perfil	Código	Frete	Tempo	CO_2
Econômico	`ECONOMICO`	× 0,85	× 1,25	× 1,00
Rápido	`RAPIDO`	× 1,40	× 0,80	× 1,15
Ecológico	`ECOLOGICO`	× 1,10	× 1,10	× 0,70

3.4 Arredondamento

- Frete, tempo e CO_2 de cada encomenda são arredondados para 2 casas decimais (`HALF_UP`) após aplicar o perfil.
- Os agregados do relatório usam os valores já arredondados por encomenda.
- A média de tempo é arredondada para 2 casas apenas no final.
- Tolerância de correção: ± 0,01 nos valores numéricos. Portanto `double` e `BigDecimal` são igualmente aceitos. O formato do texto (seção 5.2) é conferido exatamente.

4. Validação, rastreamento e notificações

4.1 Validação de encomendas

Uma encomenda inválida é rejeitada e registrada (com ID e motivo), sem lançar exceção não tratada e sem interromper o lote. Ela não entra nos totais do relatório.

Situação	Tratamento
Linha com número de campos diferente de 7, `ID` que não seja inteiro, ou peso ou distância não numéricos ou não finitos (`NaN`, `Infinity`)	Rejeitada, motivo `formato invalido`
Modal ou perfil desconhecidos	Rejeitada, motivo `codigo inexistente`
Origem ou destino vazio (após remover espaços das pontas)	Rejeitada, motivo `rota invalida`
`peso ≤ 0`	Rejeitada, motivo `peso invalido`
`distância < 0`	Rejeitada, motivo `distancia invalida`
`peso` acima da capacidade do modal (peso igual à capacidade é válido)	Rejeitada, motivo `capacidade excedida`
`distância = 0`	Válida: valem as fórmulas da seção 3.2 sem exceção (inclusive a sobretaxa), o que resulta em frete base = peso × custo por kg, tempo base = manuseio e CO_2 base = 0
Linha em branco	Ignorada: não é rejeitada e não entra em nenhuma contagem
Lista de encomendas vazia	Relatório com todos os valores iguais a 0, sem divisão por zero

Os motivos são textos fixos, em minúsculas e sem acento, exatamente como na tabela. Quando uma linha tem mais de um problema, vale o primeiro motivo da tabela, de cima para baixo (a ordem da tabela é a ordem de validação). O ID é único em cada lote: o sistema não precisa detectar IDs repetidos.

4.2 Rastreamento

O lote da seção 5 só calcula e imprime. O acompanhamento dos transportes é uma funcionalidade do núcleo, exercitada pelos testes e na defesa oral, e não tem formato de entrada. O núcleo deve oferecer, em palavras suas: criar um transporte a partir de uma encomenda válida; mudar o status; atualizar a posição; trocar o perfil; simular outro perfil (cenário da seção 7).

- **Status possíveis:** `AGUARDANDO`, `EM_TRANSITO`, `ENTREGUE`. Todo transporte nasce `AGUARDANDO` e **sem posição**.
- **Transições válidas:** `AGUARDANDO → EM_TRANSITO → ENTREGUE`. Qualquer outra transição (inclusive repetir o status atual) é rejeitada.
- **Troca de perfil:** permitida em `AGUARDANDO` e `EM_TRANSITO`. Depois de `ENTREGUE`, a troca de perfil também é rejeitada (a simulação continua permitida), para que o CO_2 já somado pela auditoria não fique inconsistente. A simulação não altera o transporte e não gera aviso.
- **Posição:** par (latitude, longitude). Latitude entre -90 e 90 e longitude entre -180 e 180, ambas finitas. Se algum valor estiver fora do intervalo, o par inteiro é rejeitado. A primeira posição atribuída conta como mudança. Atualizar a posição é permitido em qualquer status.
- **Operação rejeitada:** não altera o estado do transporte, não gera aviso a ninguém, e quem a solicitou consegue saber que foi rejeitada. O mecanismo (valor de retorno ou exceção) é decisão sua, a ser justificada no Documento de Projeto e coberta pelos testes.

4.3 Avisos automáticos

Os avisos ocorrem somente quando o status ou as coordenadas realmente mudam (atribuir o valor que já existe não é mudança). Os interessados e suas reações são:

Interessado	Reação esperada
Central de controle	Registra uma linha de log por evento (ID, status, coordenadas)
Auditoria ambiental	Ao receber `ENTREGUE`, soma o CO_2 da encomenda (com o perfil vigente nesse momento) ao acumulado auditado
Cliente	Recebe uma mensagem de texto a cada mudança de status (mudança de posição não o avisa)

No futuro, novos interessados podem surgir (veja a seção 7).

4.4 Relatório consolidado

- **Custo total:** soma dos fretes das encomendas válidas.
- **Tempo médio:** soma dos tempos dividida pela quantidade de encomendas válidas.
- **CO_2 total:** soma das emissões das encomendas válidas.
- Informar também a quantidade de encomendas válidas e de rejeitadas.

5. Entrada, saída e exemplo resolvido

5.1 Entrada (lote)

O programa lê a **entrada padrão** (`System.in`), uma encomenda por linha, campos separados por `;`, sem cabeçalho, na ordem `ID;MODAL;PESO_KG;DISTANCIA_KM;ORIGEM;DESTINO;PERFIL`. Os códigos de modal são `RODOVIARIO`, `AEREO` e `AQUAVIARIO`. A leitura termina quando não há mais linhas.

Para processar um arquivo, redirecione a entrada na execução:

```
java Main < entrada.txt
```

A leitura fica na classe de entrada (por exemplo, o `main`), que monta uma `List<String>` com as linhas e a entrega ao restante do sistema. O núcleo de cálculo não lê dados diretamente. Exemplo de leitura com `Scanner`:

```
Scanner entrada = new Scanner(System.in);
List<String> linhas = new ArrayList<>();
while (entrada.hasNextLine()) {
    linhas.add(entrada.nextLine());
}
```

Exemplo de conteúdo:

```
1;RODOVIARIO;500;1000;Curitiba;Recife;ECONOMICO
2;AEREO;500;1000;Sao Paulo;Manaus;RAPIDO
3;AQUAVIARIO;2000;3000;Santos;Belem;ECOLOGICO
```

Você decide como transformar os códigos de texto em objetos do seu modelo, respeitando as restrições da seção 6.

Regras de leitura. Valem para toda linha da entrada:

1. Divida a linha por `;` mantendo campos vazios no fim (em Java, `linha.split(";", -1)`) e remova os espaços das pontas de cada campo (`trim`).
2. Uma linha vazia, ou formada só por espaços, é uma linha em branco (ignorada, seção 4.1).
3. A linha deve ter exatamente 7 campos; senão, `formato invalido`.
4. `ID` deve ser um inteiro (`Integer.parseInt`). Peso e distância usam `Double.parseDouble` e o valor deve ser finito. Falhas dão `formato invalido`.
5. Os códigos de modal e de perfil diferenciam maiúsculas de minúsculas: `rodoviario` não é `RODOVIARIO` e dá `codigo inexistente`.
6. Origem igual ao destino é válida.
7. A base secreta usa distâncias de até 1.000.000 km e pesos finitos.

5.2 Saída

Uma linha por encomenda, na ordem de leitura, seguida de uma única linha de relatório no final. Cada linha começa com um prefixo que indica o tipo de registro. Os valores numéricos de saída usam **vírgula decimal**, duas casas e nenhum separador de milhar (por exemplo, `1445,00`). Para não depender da configuração da máquina, fixe o formato no código: `String.format(Locale.forLanguageTag("pt-BR"), "%.2f", valor)`. A banca confere exatamente o

texto (prefixos, motivos, vírgula e duas casas) e compara os valores numéricos com tolerância de ± 0,01. Os números da **entrada** (peso e distância) usam ponto decimal, como em `Double.parseDouble`.

```
VALIDA;ID;FRETE;TEMPO_H;CO2_KG
REJEITADA;ID;MOTIVO
RELATORIO;CUSTO_TOTAL;TEMPO_MEDIO_H;CO2_TOTAL;VALIDAS;REJEITADAS
```

Os três modelos acima indicam o formato; imprima apenas os dados de fato, sem cabeçalhos. Se o primeiro campo da linha rejeitada não for um inteiro legível, imprima `LINHA<n>` no lugar do ID, onde `<n>` é o número da linha física na entrada, contando também as linhas em branco (a primeira linha é 1). Exemplo completo para a entrada (a terceira linha está em branco):

```
1;RODOVIARIO;500;1000;Curitiba;Recife;ECONOMICO
2;AEREO;-5;1000;Sao Paulo;Manaus;RAPIDO

abc
3;AQUAVIARIO;2000;3000;Santos;Belem;ECOLOGICO
```

a saída é:

```
VALIDA;1;1445,00;23,33;31,00
REJEITADA;2;peso invalido
REJEITADA;LINHA4;formato invalido
VALIDA;3;2425,50;136,40;67,20
RELATORIO;3870,50;79,87;98,20;2;2
```

Os avisos de status e posição (seção 4.3) não fazem parte da saída do lote: são verificados pelos testes e na defesa oral.

5.3 Exemplo resolvido (gabarito)

Encomenda	**Frete base**	**Frete final**	**Tempo base (h)**	**Tempo final (h)**	**CO_2 base (kg)**	**CO_2 final (kg)**
1: rodoviário, 500 kg, 1000 km, `ECONOMICO`	1700,00	**1445,00**	18,67	**23,33**	31,00	**31,00**
2: aéreo, 500 kg, 1000 km, `RAPIDO`	5250,00	**7350,00**	7,25	**5,80**	301,00	**346,15**
3: aquaviário, 2000 kg, 3000 km, `ECOLOGICO`	2205,00 (já com sobretaxa de 5%)	**2425,50**	124,00	**136,40**	96,00	**67,20**

Casos de borda resolvidos (todos com perfil `ECONOMICO`; não entram no relatório abaixo):

Encomenda	**Observação**	**Frete final**	**Tempo final (h)**	**CO_2 final (kg)**
aquaviário, 2000 kg, 0 km	distância zero: a sobretaxa se aplica (300,00 × 1,05 × 0,85)	**267,75**	**30,00**	**0,00**
rodoviário, 1000 kg, 100 km	1000 kg exatos: sem sobretaxa	**467,50**	**4,58**	**6,20**

Encomenda	Observação	Frete final	Tempo final (h)	CO_2 final (kg)
rodoviário, 1001 kg, 100 km	acima de 1000 kg: com sobretaxa	**491,23**	**4,58**	**6,21**

Relatório esperado para as três encomendas:

Custo total (R$)	Tempo médio (h)	CO_2 total (kg)	Válidas	Rejeitadas
11.220,50	55,18	444,35	3	0

6. Restrições obrigatórias

Restrição técnica. Em termos de coleções, é permitido usar apenas o que foi ensinado em sala de aula. Fora isso, a forma de implementar o projeto é decisão sua.

Extensibilidade. Fora da camada de entrada, nenhum código pode testar qual modal ou qual perfil está em uso (por exemplo, com `instanceof`, `switch` ou comparação de código ou de texto). Quando um novo modal ou perfil surgir, a lógica que já existe deve continuar valendo sem ser reescrita. Você decide como alcançar isso e justifica a escolha no Documento de Projeto.

Encapsulamento. Todos os atributos de instância são `private`. Constantes `public static final` são permitidas. Getters não expõem estado mutável (use cópias defensivas ou listas não modificáveis).

7. Requisitos de qualidade e cenários de evolução

Seu projeto será avaliado pela facilidade de absorver mudanças. Os cenários abaixo serão aplicados na defesa oral e podem ser pedidos como alteração de código ao vivo. Cada alteração foi pensada para levar poucos minutos em um projeto bem estruturado; se levar muito mais, isso indica acoplamento, a ser explicado na defesa. Descreva no Documento de Projeto como seu modelo os suporta.

Cenário	Exigência
Surge o modal **ferroviário** com parâmetros próprios	Adicionar o modal criando classes novas; as únicas alterações aceitas em código existente ficam na camada de entrada
Surge um perfil **"corporativo"** com outros multiplicadores	Adicionar o perfil sem alterar o cálculo dos modais
O operador **muda o perfil ativo** de uma encomenda durante a execução	A partir da troca, os valores calculados usam o novo perfil, sem recriar o transporte
Surge um novo interessado (por exemplo, **alerta por SMS** ao atingir certa região)	Adicionar o interessado sem alterar a classe do transporte
O operador **simula** a mesma encomenda sob outro perfil, sem alterar o perfil ativo	Obter frete, tempo e CO_2 da simulação reutilizando as regras de cálculo, sem duplicá-las
O código de modal vem de **configuração externa** em vez de texto fixo	Alterar só a camada de entrada

Cenário	Exigência
Surge o status **`CANCELADO`**, alcançável a partir de `AGUARDANDO` e de `EM_TRANSITO`	Depois de `CANCELADO` nenhuma transição nem troca de perfil é aceita. A central e o cliente são avisados da mudança; a auditoria ambiental não soma nada
O **cliente cancela a assinatura** dos avisos com o sistema em execução, e outro interessado passa a recebê-los	Deixar de avisar o primeiro e avisar o segundo, sem recriar o transporte
Encomendas com peso **abaixo de 0,5 kg** passam a ser rejeitadas, com o motivo `peso minimo`	Nova validação, aplicada logo depois de `peso invalido` na ordem da tabela 4.1
A **sobretaxa** de carga pesada passa a ser de 8% acima de 2000 kg	Alterar a regra em um único ponto, para todos os modais
Surge o perfil **`PREMIUM`**: frete × 1,20, tempo × 0,90, CO_2 × 1,00, e o frete final nunca é menor que R$ 50,00	Adicionar o perfil sem alterar o cálculo dos modais nem os perfis existentes
A entrada passa a usar **`,`** como separador de campos, em vez de `;`	Alterar só a camada de entrada

Além dos cenários, o projeto deve ter **baixo acoplamento** (classes dependem de abstrações, não de implementações concretas) e **alta coesão** (cada classe tem uma responsabilidade clara).

7.1 Padrões de projeto para estudo

A lista abaixo reúne padrões GoF, em ordem alfabética, para você estudar e avaliar. Nenhum é obrigatório e nenhum é indicado para algum ponto do problema: relacionar cada padrão aos pontos de variação, ou descartá-lo, é parte da sua tarefa. Você pode usar padrões fora da lista, desde que justifique.

Padrão	Intenção (resumo)
Adapter	Converter a interface de uma classe na interface que o cliente espera
Builder	Separar a construção de um objeto complexo da sua representação
Command	Encapsular uma solicitação como objeto
Decorator	Acrescentar responsabilidades a um objeto de forma dinâmica
Factory Method / Simple Factory	Delegar a criação de objetos a um método ou a uma classe dedicados
Observer	Definir uma dependência um-para-muitos entre objetos
Prototype	Criar objetos a partir da cópia de uma instância existente
Singleton	Garantir uma única instância de uma classe, com ponto de acesso global
State	Alterar o comportamento de um objeto conforme seu estado interno
Strategy	Definir uma família de algoritmos intercambiáveis
Template Method	Definir o esqueleto de um algoritmo, adiando etapas para subclasses

Para cada padrão que usar, ou que considerar e descartar, registre no Documento de Projeto o problema que ele resolve aqui, as classes envolvidas e o motivo da escolha ou do descarte (seção 8, item 5). Usar um padrão sem justificá-lo não rende pontos.

Referência: FREEMAN, E.; FREEMAN, E.; SIERRA, K.; BATES, B. *Use a cabeça! padrões de projetos*. 2. ed. rev. Rio de Janeiro: Alta Books, 2009.

8. Etapa 1: Documento de Projeto (antes do código)

Prazo: 27/11. Nesta etapa entregue apenas a documentação, sem código. O professor vai usá-la para analisar a estratégia que você adotou.

Entregue o Documento de Projeto **antes** de implementar. Ele fixa suas decisões e será comparado com o código final. Mudanças de rumo durante a implementação são normais, mas devem ser registradas e justificadas na Etapa 2.

O documento deve conter:

1. **Análise do problema.** Liste os conceitos que você identificou (substantivos e verbos relevantes do enunciado) e indique quais viraram classes, quais viraram atributos e quais foram descartados, com o motivo.
2. **Catálogo de classes e interfaces.** Para cada uma, informe nome, responsabilidade em uma frase, principais atributos e métodos, e com quem colabora. Use o formato de cartões CRC ou uma tabela equivalente.
3. **Diagrama de classes UML** com as hierarquias, as interfaces e as dependências. Recomendado para a apresentação; pode ser feito em ferramenta (por exemplo, Mermaid ou PlantUML).
4. **Estratégia de resolução.** Explique como o projeto lida com cada ponto de variação do problema: o que muda entre os modais, entre os perfis de precificação e entre os interessados nas notificações, e qual mecanismo você escolheu para isolar cada variação.
5. **Padrões e mecanismos escolhidos.** Para cada padrão de projeto (GoF ou outro) que você pretende usar, informe o problema que ele resolve aqui, as classes que o compõem e uma alternativa que você considerou e descartou. Se decidir **não** usar um padrão em algum ponto, justifique.
6. **Fluxos principais.** Mostre em diagrama de sequência ou em texto passo a passo: (a) processamento de uma linha do lote, (b) troca de perfil em tempo de execução, (c) mudança de status de um transporte e os avisos resultantes.
7. **Resposta aos cenários de evolução** da seção 7: para cada um, diga quais classes seriam criadas e quais seriam alteradas.
8. **Plano de testes**: casos que você pretende cobrir, incluindo os de borda da seção 4.1.

9. Etapa 2: implementação e entregáveis finais

Prazo: 04/12, dia da apresentação. A implementação é mostrada no próprio dia, junto com o stress test e a defesa oral (seção 10).

Implemente em Java o projeto da Etapa 1. Entregue:

1. **Código-fonte** completo e compilável, com instruções de compilação e execução. A execução deve funcionar com entrada redirecionada (`java Main < entrada.txt`). Se o projeto usar pacotes, entregue um arquivo `COMO_EXECUTAR.md` com a versão do JDK, a classe principal e os comandos exatos de compilação e execução.
2. **Testes unitários** cobrindo os cálculos (incluindo o gabarito da seção 5.3), os casos de borda da seção 4.1, a troca de perfil em tempo de execução e os avisos automáticos.

3. **Documento de Projeto revisado**, com uma seção "Do projeto ao código" que lista cada divergência em relação à Etapa 1 e o motivo.
4. **Relatório de uso de IA** (máximo de 3 páginas) contendo:
5. os principais prompts utilizados, incluindo como você descreveu o projeto à ferramenta;
6. pelo menos **3 casos em que a IA errou** (violou uma restrição, propôs um modelo acoplado ou gerou código incorreto) e como você identificou e corrigiu cada um;
7. pelo menos **1 caso em que você rejeitou uma sugestão de projeto da IA** e manteve a sua, com o motivo.

10. Stress test, defesa oral e integridade acadêmica

10.1 Stress test final

No encerramento, os projetos rodam contra uma **base de dados secreta em lote**, entregue pela entrada padrão (a banca executa o projeto seguindo o `COMO_EXECUTAR.md` do aluno, com `< base.txt`), com um volume reduzido de encomendas e casos de borda (peso zero, distância zero, distâncias grandes, listas vazias, entradas inválidas). Serão observados:

- **eficiência:** relatório em uma única passagem pela lista (O(n)), sem varreduras aninhadas nem ordenações desnecessárias;
- **robustez:** nenhuma exceção não tratada; linhas inválidas nunca interrompem o lote;
- **consistência:** texto no formato da seção 5.2 e valores iguais ao gabarito, dentro de ± 0,01.

10.2 Defesa oral

Cada aluno ou grupo defende o projeto em arguição individual. Você poderá ser solicitado a:

- explicar qualquer decisão do Documento de Projeto e defender uma alternativa que descartou;
- implementar ao vivo um dos cenários de evolução da seção 7 (alteração de poucos minutos);
- apontar, no seu código, onde cada padrão escolhido aparece.

10.3 Integridade acadêmica

O uso de IA é permitido e esperado, mas **você é responsável por todo o projeto e todo o código entregue**. A incapacidade de explicar ou de modificar o que foi entregue reduz a nota.